



Lista de Exercícios Dependabilidade e Teste de Software

Prof. Iaçanã Ianiski Weber

Parte 1 — Conceitos de Dependabilidade

Exercício 1. Marque a alternativa que descreve corretamente a cadeia *fault* $\rightarrow$ *error* $\rightarrow$ *failure*:

- (A) Um *fault* é sempre detectável imediatamente; um *error* é o estado incorreto resultante; uma *failure* é o defeito presente no código-fonte.
- (B) Um *fault* é um defeito latente no sistema; um *error* é um estado incorreto ativado pelo *fault*; uma *failure* ocorre quando o *error* compromete o serviço entregue ao usuário.
- (C) Uma *failure* precede o *error*, que por sua vez ativa o *fault* durante a execução.
- (D) *Fault* e *failure* são sinônimos; *error* é o mecanismo de detecção utilizado para identificá-los em tempo de execução.

Exercício 2. Considere o seguinte cenário: um microsserviço de pagamentos processa uma requisição de débito, mas um bug faz com que o saldo seja decrementado duas vezes quando ocorre um timeout seguido de retry pelo cliente.

- a) Identifique o *fault*, o *error* e a *failure* neste cenário.
- b) Proponha um mecanismo para evitar o problema (idempotência, deduplicação ou outro).

Parte 2 — Confiabilidade e Disponibilidade

Exercício 3. Um servidor operou continuamente por 60 dias. Nesse período, ocorreram 4 falhas. Os tempos de reparo foram: 2,0h; 1,0h; 3,0h; 0,5h.

- a) Calcule o MTBF (em horas).
- b) Calcule o MTTR.
- c) Calcule a disponibilidade A.
- d) Estime o *downtime* anual desse sistema.

Exercício 4. Considere dois sistemas reparáveis:

- Sistema P: $MTBF = 800$ h, $MTTR = 16$ h
- Sistema Q: $MTBF = 200$ h, $MTTR = 1$ h

- a) Calcule a disponibilidade de cada sistema.
- b) Qual sistema é mais **confiável**? E qual é mais **disponível**?
- c) Explique, em termos práticos, por que confiabilidade e disponibilidade podem apontar para sistemas diferentes.

Exercício 5. Assumindo o modelo exponencial de falhas com $MTBF = 1000$ h:

- a) Qual a taxa de falha λ ?
- b) Calcule $R(200)$, ou seja, a probabilidade de operar sem falha por 200 horas.
- c) Calcule $R(1000)$.
- d) Se a missão exige confiabilidade de pelo menos 95% em 100 horas, esse sistema atende?

Exercício 6. Uma empresa deseja garantir um SLA de $A \geq 99,9\%$ para sua plataforma. O sistema atual tem $MTBF = 500$ h.

- a) Qual deve ser o $MTTR$ máximo para atender esse SLA?
- b) Com esse $MTTR$ máximo, estime o *downtime* anual permitido.
- c) Se na prática o $MTTR = 2$ h, o SLA é atendido? Justifique.

Exercício 7. Indique **V** (verdadeiro) ou **F** (falso) para cada afirmação sobre MTTF e MTBF:

- () O MTTF é utilizado em sistemas não reparáveis, enquanto o MTBF é a métrica adequada para sistemas reparáveis.
- () $MTBF = MTTF$, pois ambos medem o tempo médio até a primeira falha do sistema.
- () Para um disco rígido de servidor que pode ser substituído, o MTBF é a métrica mais apropriada.
- () Um MTBF alto implica necessariamente alta disponibilidade, independentemente do valor do MTTR.

Exercício 8. Um sistema deve operar continuamente por 48 horas para uma missão crítica. Duas opções estão disponíveis:

- Opção A: $MTBF = 400$ h
- Opção B: $MTBF = 1200$ h

- a) Calcule $R(48)$ para cada opção (modelo exponencial).
- b) Qual o ganho em pontos percentuais de confiabilidade da opção B sobre a opção A?
- c) Se o custo da opção B é $3\times$ maior, você recomendaria a troca? Justifique considerando o contexto de missão crítica.

Parte 3 — Safety, Security e Resiliência

Exercício 9. Marque a alternativa que diferencia corretamente *safety* e *security* no contexto de dependabilidade:

- (A) *Safety* refere-se à proteção contra ameaças intencionais de agentes externos; *security* trata de falhas acidentais que podem causar danos ao ambiente.
- (B) *Safety* preocupa-se com a proteção do ambiente contra consequências catastróficas de falhas acidentais do sistema; *security* protege o sistema contra ameaças e ações maliciosas intencionais.
- (C) *Safety* e *security* são sinônimos no contexto de dependabilidade, ambos referindo-se à ausência de consequências catastróficas.
- (D) *Security* é um subconjunto de *safety*, pois toda vulnerabilidade de segurança representa também um risco de *safety*.

Exercício 10. Considere um sistema de controle de tráfego ferroviário. Um atacante compromete a rede de comunicação e injeta comandos falsos de sinalização.

- a) Esse cenário é primariamente um problema de *safety* ou de *security*? Justifique.
- b) Explique como uma vulnerabilidade de *security* pode se tornar um risco de *safety*.
- c) Proponha pelo menos duas mitigações que enderecem tanto o aspecto de *safety* quanto o de *security*.

Exercício 11. Explique o conceito de *hazard* e diferencie-o de *failure*. Em seguida, para um sistema de piloto automático de drone de entrega, identifique:

- a) Dois possíveis *hazards*.
- b) Para cada *hazard*, uma *safety constraint* (restrição de segurança funcional).
- c) Uma mitigação arquitetural para cada *hazard*.

Exercício 12. Indique **V** (verdadeiro) ou **F** (falso) para cada afirmação sobre as propriedades CIA e AAA:

- () Confidencialidade (CIA) é violada quando dados de pacientes são acessados por pessoal não autorizado de outro setor hospitalar.
- () Integridade (CIA) refere-se à disponibilidade contínua do sistema de prontuário, garantindo acesso ininterrupto 24 horas por dia.
- () Disponibilidade (CIA) é violada quando um ataque de negação de serviço (DoS) torna o sistema de prontuário inacessível.
- () Autenticação (AAA) determina quais recursos e operações um usuário já identificado pode executar no sistema.
- () Accountability (AAA) permite rastrear quem acessou ou modificou um prontuário eletrônico e em que momento.

Exercício 13. Explique por que projetar *safety* e *security* separadamente pode ser arriscado. Dê um exemplo concreto em que uma mitigação de *security* cria um novo risco de *safety* (ou vice-versa).

Parte 4 — Teste de Software: Caixa Preta

Exercício 14. Uma função recebe um inteiro `idade` e retorna uma string conforme as regras:

- Se $0 \leq \text{idade} \leq 12$: retorna “criança”
- Se $13 \leq \text{idade} \leq 17$: retorna “adolescente”
- Se $18 \leq \text{idade} \leq 64$: retorna “adulto”
- Se $65 \leq \text{idade} \leq 150$: retorna “idoso”

- a) Identifique todas as classes de equivalência (válidas e inválidas).
- b) Projete um caso de teste representativo para cada classe.
- c) Identifique todas as fronteiras e projete casos de teste para os valores *on* e *off* de cada fronteira.

Exercício 15. Considere a seguinte especificação de uma função que calcula o desconto em uma loja online:

```
float calcular_desconto(float valor, int tipo_cliente, int cupom);
```

Parâmetro	Tipo	Domínio	Descrição
valor	float	(0, 10000]	Valor da compra em reais
tipo_cliente	int	{1, 2, 3}	1=normal, 2=premium, 3=VIP
cupom	int	{0, 1}	0=sem cupom, 1=com cupom

Regras:

- Se qualquer parâmetro estiver fora do domínio, retornar -1 .
- Desconto base: normal = 0%, premium = 10%, VIP = 20%.
- Se `cupom = 1`, adicionar 5% ao desconto.
- Desconto máximo: 25%. Se o desconto calculado ultrapassar 25%, limitar a 25%.
- Retorno: valor do desconto em reais ($\text{valor} \times \text{percentual}$).

- a) Identifique todas as classes de equivalência inválidas.
- b) Identifique todas as classes de equivalência válidas (considere as combinações de tipo de cliente e cupom).
- c) Identifique as fronteiras relevantes e projete casos de teste para valores *on* e *off*.
- d) Monte uma suíte de testes consolidada em tabela com colunas: ID, valor, tipo_cliente, cupom, resultado esperado, técnica, classe coberta.

Exercício 16. Para a especificação do Exercício 15, identifique pelo menos 3 defeitos plausíveis que um programador poderia introduzir na implementação. Para cada defeito, indique qual caso de teste da sua suíte o detectaria.

Exercício 17. Uma função recebe três inteiros representando os lados de um triângulo e classifica-o como equilátero, isósceles, escaleno ou erro (não forma triângulo). Sabe-se que a implementação contém o seguinte defeito: a condição de desigualdade triangular usa $\geq$ em vez de $>$ (ou seja, $a+b \geq c$ em vez de $a+b > c$).

- a) Qual caso de teste detecta esse defeito? Forneça valores concretos.
- b) Explique por que um caso de teste com valores “genéricos” (ex.: $a = 3, b = 4, c = 5$) **não** detectaria esse defeito.
- c) Qual técnica de teste (particionamento ou valor limite) é mais eficaz para encontrar esse tipo de bug?

Parte 5 — Teste de Software: Caixa Branca

Exercício 18. Considere o seguinte código em C:

```
1  int verificar_acesso(int nivel, int autenticado) {  
2      int resultado = 0;  
3      if (nivel >= 3 && autenticado == 1) {  
4          resultado = 1;  
5      }  
6      if (nivel >= 5) {  
7          resultado = 2;  
8      }  
9      return resultado;  
10 }
```

a) Desenhe o grafo de fluxo de controle (CFG) deste código.

Exercício 19. Considere o seguinte código:

```
1  int calcular_bonus(int vendas, int tempo_empresa) {  
2      int bonus = 0;  
3      if (vendas > 100 || tempo_empresa > 5) {  
4          bonus = 500;  
5          if (vendas > 200 && tempo_empresa > 10) {  
6              bonus = 1500;  
7          }  
8      }  
9      return bonus;  
10 }
```

a) Desenhe o grafo de fluxo de controle (CFG).

b) É possível alcançar 100% de cobertura de caminhos com menos de 4 casos de teste? Justifique.

Exercício 20. Considere o seguinte código:

```
1 int processar(int a, int b) {  
2     int r = a;  
3     if (a > b) {  
4         r = a - b;  
5     } else {  
6         r = b - a;  
7     }  
8     if (r == 0) {  
9         r = -1;  
10    }  
11    return r;  
12 }
```

- a) Identifique todos os caminhos possíveis no código.
- b) Projete casos de teste para cobertura de caminhos (path coverage) de 100%.
- c) Existe algum caminho infactível? Se sim, qual e por quê?